

Problem Set 3

Algoritmi e Strutture Dati – a.a. 2025/2026

Università di Roma “Tor Vergata”

Prof. Luciano Gualà

Importanti avvertenze preliminari:

1. Il gruppo deve essere formato da almeno tre persone!
2. Per lo svolgimento si consiglia fortemente l'utilizzo del template L^AT_EX disponibile nella pagina web del corso.
3. Leggere il documento *Problem Set: Istruzioni e consigli* sulla pagina web del corso.

Linee guida per la consegna. Gli elaborati dovranno essere consegnati entro domenica 31/05/2026 alle ore 23:59. Si prega di seguire le seguenti indicazioni:

1. le soluzioni dovranno essere consegnate via email a entrambi gli indirizzi `guala@mat.uniroma2.it` e `alessandrostr95@gmail.com`;
2. l'oggetto dell'email dovrà essere “Consegna problem set 3 - 2025/2026”;
3. il file dovrà essere in formato pdf;
4. il nome del file dovrà essere “PS_3_XXX.pdf” dove XXX è il primo cognome (in lower case) in ordine alfabetico dei membri del gruppo;
5. l'email dovrà essere inviata tramite l'indirizzo dello studente XXX presente nel nome del file;
6. inserire nel file nome, cognome, indirizzo email e matricola di tutti i membri del gruppo (meglio ancora se tutti i membri del gruppo sono in indirizzo nell'email della consegna).

Buon lavoro 🍌👓!

Problema 1 (*Il dottor maestro all'All you can eat*)

Grazie al vostro aiuto nel precedente problem set (vedi Problema 1), il dottor maestro Straziota è finalmente riuscito ad arrivare in Oriente senza problemi. Per festeggiare, decide di andare a mangiare sushi in una famosa catena chiamata Sushiro.

Nel ristorante, i piatti scorrono lungo i tavoli tramite un nastro trasportatore. Esistono due tipi di piatti: i piatti blu, i quali contengono un solo pezzo di sushi, e i piatti rossi, i quali contengono due pezzi di sushi.

Più precisamente, il nastro trasportatore funziona nel seguente modo. Il tempo è suddiviso in N istanti di tempo discreti, e ad ogni istante di tempo:

- il nastro avanza di una posizione;
- un nuovo piatto può comparire davanti al dottor Straziota;
- il piatto corrente viene portato via se non viene preso.

Secondo la tradizione giapponese, un pezzo di sushi *deve* essere mangiato in un solo boccone. Tuttavia, nonostante Alessandro sia veloce a mangiare, impiega comunque esattamente 1 istante di tempo per mangiare un pezzo di sushi.

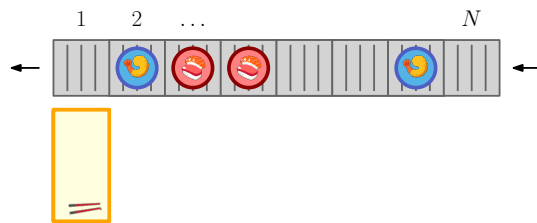
Il tavolo non è molto spazioso e può contenere al massimo due piatti contemporaneamente. Quando Alessandro svuota un piatto, però, può impilarvi sopra un nuovo piatto (pieno) preso dal nastro.

Inoltre, durante ogni istante di tempo il dottor Straziota può compiere *una sola* delle seguenti azioni:

- a) prendere il piatto attualmente disponibile sul nastro (solo se sul tavolo c'è spazio sufficiente);
- b) mangiare un pezzo di sushi da uno dei piatti presenti sul tavolo.

Di conseguenza, svuotare un piatto blu richiede 1 istante di tempo, mentre svuotare un piatto rosso ne richiede 2.

Lungo il nastro scorrono complessivamente $n \leq N$ piatti, e immaginate che in input avete la sequenza degli n piatti (ordinata per posizione), dove l' i -esimo elemento della sequenza specifica il colore c_i dell' i -esimo piatto, e la sua posizione p_i sul nastro. Progettate un algoritmo di programmazione dinamica che determini in tempo $O(n)$ qual è il numero massimo di pezzi di sushi che il dottor maestro Straziota può mangiare.



(a) Un esempio di istanza con $N = 8$ ed $n = 4$. L'istanza consiste nella sequenza $(c_1 = B, p_1 = 2), (c_2 = R, p_2 = 3), (c_3 = R, p_3 = 4), (c_4 = B, p_4 = 7)$. Il maggior numero di pezzi di sushi che si può mangiare è 5, prendendo i due piatti rossi in posizione 3 e 4, e l'ultimo piatto blu in posizione 7. Gli istanti di tempo 5 e 6 sono utilizzati da Straziota per mangiare due pezzi di sushi da uno stesso piattino rosso, in modo che possa poi impilare il piattino blu in posizione 7 sul piatto rosso vuoto. Gli istanti di tempo 8, 9 e 10 sono utilizzati per mangiare i 3 pezzi di sushi che sono sul tavolo, mentre il nastro ormai scorre vuoto davanti a Straziota.



(b) Il dottor Straziota che cerca di mangiare quanto più sushi al Sushiro di Hong Kong (Sheung Wan).

Figure 1

Problema 2 (*Festa aziendale a basso budget*)

Sei stato incaricato di organizzare la festa annuale della tua azienda. La direzione ha imposto una regola fondamentale: *tutti gli invitati devono divertirsi!*

Purtroppo, l'azienda ha una struttura gerarchica estremamente rigida, rappresentata da un albero radicato. Per evitare situazioni imbarazzanti, non è possibile invitare contemporaneamente un dipendente e il suo diretto superiore. In tal caso, infatti, nessuno dei due riuscirebbe a rilassarsi davvero.

Ogni dipendente v possiede inoltre un *fun score* $\sigma(v)$, ovvero un numero intero che misura quanto quella persona contribuisca al divertimento complessivo della festa.

Per rendere la festa il più possibile memorabile¹, il tuo obiettivo è massimizzare la somma dei fun score degli invitati.

Tuttavia, a causa dei tagli al budget, l'azienda ha imposto un ulteriore vincolo: puoi invitare al massimo k dipendenti!

Più formalmente, l'azienda è rappresentata da un albero radicato pesato $T = (V, E, \sigma : V \rightarrow \mathbb{N})$.

Progettare un algoritmo efficiente che determini un sottoinsieme di invitati $S \subseteq V$ tale che:

1. $|S| \leq k$;
2. per ogni dipendente $v \in S$, il suo diretto superiore non appartiene a S ;
3. la somma dei fun score $\sum_{v \in S} \sigma(v)$ degli invitati è *massimizzata*.

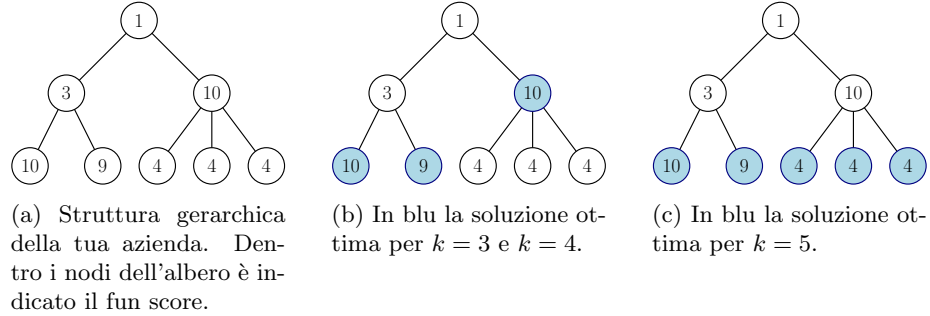


Figure 2

¹E sperare così in una promozione.

Problema 3 (*Il Babbo Natale algoritmista*)

Pochi sanno che Babbo Natale ha una laurea in informatica e ancora meno persone sospettano quanto gli siano utili le cose imparate nelle aule di SoGeNe.² Per esempio, ogni anno Babbo Natale deve affrontare un problema di ottimizzazione da cui dipende in una certa misura la bellezza del mondo: rendere felici il maggior numero di bambini.

Un'istanza del problema può essere descritta nel seguente modo. Babbo Natale ha un sacco enorme pieno di giocattoli, che per semplicità sono numerati da 1 a N . Nel mondo vivono k bambini e ogni bambino ha una sua propria lista di desideri. La lista dei desideri del bambino i – che di solito Babbo Natale riceve in forma di lettera – è un sottoinsieme dei giocattoli S_i . I giocattoli sono indivisibili, e se Babbo Natale decide di dare il giocattolo j ad un certo bambino non può darlo a nessun altro.

L'obiettivo di Babbo Natale è fare le consegne con la slitta il 24 dicembre in modo da massimizzare il numero di bambini che saranno contenti la mattina del 25 quando scarteranno i regali che troveranno sotto l'albero.

Il problema ha almeno due varianti.

- Fino a non molto tempo fa, infatti, un bambino era felice se riceveva anche *uno solo* dei giocattoli della sua lista di desideri.
- Ultimamente, però, i bambini sono diventati più esigenti³ e un bambino oggi resta deluso se non riceve *tutti* i giocattoli della sua lista dei desideri.

Quale variante è computazionalmente più difficile? Quale ammette un algoritmo veloce? Può Babbo Natale massimizzare il numero di bambini felici in tempo polinomiale?

²La leggenda narra che anche a quei tempi esistesse già SoGeNe e che il colore del bar che ha riaperto recentemente fosse stato scelto proprio in onore di uno studente panciuto che anche allora vestiva sempre di rosso.

³Se non avesse paura dell'Associazione delle mamme, Babbo Natale userebbe la parola *viziati*.